

Fast Track — Mock Exam 3 (Solutions)

PayFlow scenario · full worked solutions

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

How to use these solutions

- One slide per question, in exam order: **Part A** (Python) → **Part E** (production).
- Each slide restates the task, then gives the full worked answer; code is runnable with modern `pandas` / `scikit-learn`.
- For MCQ we give the **correct key** and say why each distractor is wrong.
- Scenario: **PayFlow**, a fintech — transactions, fraud, KYC, transfers. 100 marks, 5 parts of 20.
- **Green** = correct / do this; **red** = the trap.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Reconciling a refund, PayFlow evaluates `int(-3.9)`. Result?

Answer: B) -3.

- `int()` **truncates toward zero** — it drops the fractional part, it does not round. So `int(-3.9) == -3` and `int(3.9) == 3`.
- A) `-4` would be *rounding* / `math.floor(-3.9)`. C) `0` is nonsense. D) `4` confuses sign and rounds.
- To round properly use `round(-3.9) == -4`; for a direction use `math.floor` / `math.ceil`.

`size = requested or 50` with `requested = 0`. What is `size`?

Answer: B) 50.

- `x or default` keys off *truthiness*, and 0 is *falsy*. So `0 or 50` evaluates to 50 — the valid value 0 is silently discarded.
- A) 0 is what you *wanted* but not what the code does. C)/D) no `None`, no error is produced.
- Fix when 0 is legal: `size = 50 if requested is None else requested` — test presence with `is None`, not truthiness.

```
def add_txn(txn, batch=[]): # BUG: default list created ONCE
    batch.append(txn)
    return batch
```

What goes wrong: the default `[]` is built a single time when the `def` runs, and *shared* by every call that omits `batch`. Successive "different" batches all append to the same list, so transactions leak across calls.

```
def add_txn(txn, batch=None): # sentinel default
    if batch is None:
        batch = [] # fresh list per call
    batch.append(txn)
    return batch
```

```
class Wallet:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("insufficient funds")
        self.balance -= amount

    def statement(self):
        return f"{self.owner}: {self.balance:.2f} EUR"
```

Marks: constructor with default (1), deposit (1), withdraw with the ValueError guard (2), formatted statement (1).

```
top = amounts.sort(reverse=True) # BUG
print(top[:3]) # TypeError: NoneType not subscriptable
```

Root cause: `list.sort()` sorts the list *in place* and **returns None**. So `top` is `None`, and `None[:3]` raises. (Same trap for `.append`, `.reverse`, `.extend`.)

Fix — use `sorted()`, which returns a *new* list:

```
top = sorted(amounts, reverse=True)
print(top[:3]) # [340.5, 210.0, 12.0]
```

```
from collections import Counter

counts = Counter(t["category"] for t in txns) # comprehension -> Counter
# counts -> Counter({'groceries': 2, 'travel': 1, ...})

top_category = counts.most_common(1)[0][0] # most frequent name
```

- Counter consumes any iterable; a generator expression avoids building an intermediate list.
- `most_common(1)` returns `[(name, count)]`; index `[0][0]` pulls the name.
- Equivalent manual tally: `d[k] = d.get(k, 0) + 1` in a loop.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Select amount > 1000 **and** is_fraud == 1.

Answer: B) `df[(df.amount>1000) & (df.is_fraud==1)]`.

- Element-wise combination needs the **bitwise** & (or |), and each condition must be **parenthesised** because & binds tighter than the comparisons.
- A) uses Python and on Series → `ValueError: ambiguous truth value`.
- C) drops the parentheses, so `1000 & df.is_fraud` is evaluated first → error.
- D) uses or (same ambiguity) and the wrong logic.

Keep only merchants whose *total* volume exceeds 1,000,000.

Answer: B) `GROUP BY merchant HAVING SUM(amount) > 1000000`.

- WHERE filters *rows* before grouping and cannot see aggregates; a per-group aggregate condition belongs in HAVING (which runs after GROUP BY).
- A) puts WHERE after GROUP BY *and* references SUM in WHERE — both illegal.
- C) filters individual big transactions, not merchant totals. D) is not a filter at all.

```
scorecard = transactions.groupby("merchant").agg(  
    n_txns=("amount", "count"),  
    total_amount=("amount", "sum"),  
    fraud_rate=("is_fraud", "mean"), # mean of a 0/1 column = a rate  
)
```

- **Named aggregation:** each output column is `name=(source_column, function)`.
- The key idea: the **mean of a 0/1 indicator column is its rate** — `fraud_rate` needs no manual division.
- Result is indexed by `merchant`; add `.reset_index()` for a flat frame.

- **(a)** `scipy.stats.ttest_ind(a, b, equal_var=False)` — `equal_var=False` makes it **Welch's** t-test (unequal variances); read `.pvalue`.
- **(b)** *If there were truly no difference between the models*, we would see a difference in means at least this large about **3%** of the time. Since $0.03 < 0.05$ we reject H_0 at the 5% level. It is **not** "a 97% chance B is better" and not the probability that H_0 is true.
- **(c)** The p-value shows *whether* there is a signal, not *how big* it is — with enough reviewers even a trivial gap turns "significant". **Cohen's d** reports the standardised effect size, i.e. the practical magnitude PayFlow actually cares about.

```
transactions.groupby("merchant")["amount"].mean("amount") # TypeError
```

Mistake: after `["amount"]` you have already selected a single Series. `Series/GroupBy.mean()` takes no column name — its first positional argument is `axis`, and passing the string `"amount"` is invalid.

Fix — call `.mean()` with no argument:

```
transactions.groupby("merchant")["amount"].mean()
```

```
SELECT m.name,  
       COUNT(*) AS n_completed,  
       SUM(t.amount) AS total_amount  
FROM transactions AS t  
JOIN merchants AS m ON m.merchant_id = t.merchant_id  
WHERE m.country = 'DE'  
      AND t.status = 'completed'  
GROUP BY m.name  
HAVING COUNT(*) > 100  
ORDER BY total_amount DESC;
```

- Row filters (country, status) go in WHERE; the per-merchant count filter goes in HAVING.
- JOIN on merchant_id; group by the displayed name; ORDER BY may use the SELECT alias.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering**
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Predicting fraud *at authorisation time*. Which feature leaks?

Answer: C) `chargeback_filed`.

- The golden rule: a feature may only use information available **at prediction time**. A chargeback is logged *days after* the transaction, so it is future knowledge — it would inflate offline scores, then vanish in production.
- A) amount, B) merchant country, D) hour of day are all known the instant the transaction is authorised — legitimate features.

0.5% fraud; model always predicts "not fraud".

Answer: B) ~99.5% accuracy, catches zero fraud; report PR-AUC.

- Predicting the majority class scores $1 - 0.005 = 99.5\%$ accuracy while catching *no* fraud — accuracy is meaningless here.
- Use **PR-AUC** = `average_precision_score(y, score)`; the no-skill baseline is the fraud *rate* (0.005).
- A) accuracy is not 50%. C) high accuracy does *not* prove skill. D) accuracy is not 0.5%.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
```

```
prep = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
model = Pipeline([
    ("prep", prep),
    ("clf", LogisticRegression(max_iter=1000)),
])
model.fit(X_train, y_train)
```

`handle_unknown="ignore"` tolerates unseen categories at predict time. Putting the scaler *inside* the pipeline fits it on train folds only — no leakage during CV.

```
import numpy as np
from sklearn.metrics import confusion_matrix
COST_FP, COST_FN = 5, 200

def total_cost(y_true, proba, t):
    pred = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()
    return fp * COST_FP + fn * COST_FN

thresholds = np.linspace(0.05, 0.95, 91)
costs = [total_cost(y_test, proba, t) for t in thresholds]
best_t = thresholds[int(np.argmin(costs))]
```

Why not 0.5: the costs are *asymmetric* — a missed fraud costs 40× a false alarm, so the money-minimising cutoff sits *well below* 0.5 (flag more aggressively). A default 0.5 optimises accuracy, not euros.

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred) # BUG 1
recall = tp / (tp + fp) # BUG 2
```

- **Bug 1:** `confusion_matrix` returns a 2×2 array, not four scalars — unpacking fails. Flatten it with `.ravel()`.
- **Bug 2:** `tp/(tp+fp)` is *precision*. Recall is `tp/(tp+fn)`.

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
recall = tp / (tp + fn)
```

```
import numpy as np
def precision_at_k(y_true, scores, k):
    idx = np.argsort(-scores)[:k] # k riskiest
    return np.asarray(y_true)[idx].mean()

def recall_at_k(y_true, scores, k):
    idx = np.argsort(-scores)[:k]
    caught = np.asarray(y_true)[idx].sum()
    return caught / np.asarray(y_true).sum()
```

(b) With 35 frauds in the top 50 and 200 frauds total:

$$\text{precision@50} = \frac{35}{50} = 0.70, \quad \text{recall@50} = \frac{35}{200} = 0.175.$$

Shrinking k raises precision but lowers recall — the review-queue trade-off.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering**
- 5 Part E — Applied & production

Which MCP primitive is **model-controlled**?

Answer: C) tools.

- **Tools** are *model-controlled* — the model chooses when to call them (e.g. "look up this transaction's risk").
- **Resources** (A) are *application-controlled* — a read-only GET addressed by URI, loaded by the host app.
- **Prompts** (B) are *user-controlled* message templates the user invokes.
- D) "sampling" is a server→client request, not one of the three core primitives.

Primary purpose of `max_steps` in an agent loop?

Answer: B) a fuse that stops a runaway / infinite tool-calling loop.

- An agent can loop forever (call tool → observe → call again). `max_steps` caps the iterations and returns a "stopped: budget reached" result — the single most important safety parameter.
- A) it does not change call quality. C) temperature is separate. D) it bounds *steps*, not the number of registered tools.

```
import re, json

def safe_json(text):
    cleaned = re.sub(r"^```(?:json)?\s*|\s*```$", "",
                    text.strip(), flags=re.I | re.M)
    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        return {"error": "unparseable", "raw": text}
```

- Strip a leading "``` / ```json and a trailing ``` fence, then parse.
- Catch *only* json.JSONDecodeError and return a fallback dict — never let a bad model response raise. The parser is your seatbelt; a tighter prompt is the real fix.

Two documented biases, each with a mitigation:

- **Verbosity (length) bias** — the judge tends to score *longer* answers higher regardless of quality. *Mitigation*: give the rubric explicit length-independent criteria and/or normalise for length; cross-check with a different (e.g. smaller) judge model.
- **Position (order) bias** — when comparing two answers, the judge favours whichever is presented *first*. *Mitigation*: run both orderings (A,B) and (B,A) and average, so position cancels out.

(Also acceptable: self-preference / style bias; mitigate by validating the judge against human labels and requiring $\geq 80\%$ agreement before trusting it.)

One prose (non-JSON) response kills the whole loop. Wrap the parse:

```
import json
records = []
for row in receipts:
    raw = llm_extract(row)
    try:
        records.append(json.loads(raw))
    except json.JSONDecodeError:
        records.append({"error": "bad_json", "raw": raw}) # flag, keep going
```

Rule: never call `json.loads` unguarded on model output. Catch `JSONDecodeError` per row so one malformed receipt is flagged (or routed to a human) instead of aborting the batch.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vec = TfidfVectorizer(stop_words="english")
doc_vectors = vec.fit_transform(TEXTS) # fit on the corpus
q_vec = vec.transform([query]) # transform, do NOT re-fit
sims = cosine_similarity(q_vec, doc_vectors)[0]
top3 = np.argsort(-sims)[:3] # indices of 3 best docs
```

(b) Cosine is $\frac{a \cdot b}{\|a\| \|b\|}$. If the vectors are **L2-normalised** then $\|a\| = \|b\| = 1$, so the denominator is 1 and cosine similarity reduces to the plain dot product $a \cdot b$ — cheaper to compute and rank-equivalent.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Why not default `train_test_split` on a daily time series?

Answer: B) default `shuffle=True` mixes future rows into training — leakage.

- A random shuffle puts *future* days in the training set and past days in test, so the model "sees the future" — scores look great offline, fail live.
- Split by *time* instead: `train = s.iloc[:n]; test = s.iloc[n:]` (or `shuffle=False` / `TimeSeriesSplit`).
- A) speed is irrelevant. C) stratify is for classification. D) the index is not dropped.

Before scraping a regulator's rates page, you should:

Answer: B) check for an official API / data export first; scrape static HTML only as a fallback.

- The rule is "check for an API first": an API/export is more stable, cleaner, and usually explicitly permitted. Scraping is the *last* rung of the ladder.
- A) ignores rate limits / politeness. C) a headless browser is overkill for static HTML. D) robots.txt and Terms of Service still apply to public pages.

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

fit = ExponentialSmoothing(
    train, trend="add", seasonal="add", seasonal_periods=7,
    initialization_method="estimated",
).fit()

fc = fit.forecast(h) # h steps ahead
mae = (test - fc).abs().mean() # mean absolute error
```

- `seasonal_periods=7` = weekly cycle; additive trend + additive seasonal = "triple exponential smoothing".
- `forecast(h)` extends past the training window; align test and fc on the same index.
- $MAE = \frac{1}{n} \sum |y - \hat{y}|$; compare against a naive/seasonal-naive baseline to judge lift.

```
import time, functools

def retry(attempts=3, base_delay=0.5, backoff=2.0):
    def decorator(func):
        @functools.wraps(func) # preserve name/docstring
        def wrapper(*args, **kwargs):
            delay = base_delay
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    if attempt == attempts: # last try: give up
                        raise
                    time.sleep(delay)
                    delay *= backoff # exponential
            return wrapper
        return decorator
```

Three nested layers (args → func → call). Re-raise on the final attempt. In production: cap the delay and add jitter, and only retry *transient* errors.

```
title = soup.find("span", class_="merchant").text # AttributeError if missing
```

Cause: `find` returns `None` when no matching tag exists, and `None.text` raises. Capture the node, test it, then read:

```
node = soup.find("span", class_="merchant")
title = node.get_text(strip=True) if node else None
```

`get_text(strip=True)` also trims whitespace. Same discipline for `select_one` / attribute access at any scraping boundary.

```
from urllib import robotparser
from bs4 import BeautifulSoup
import requests

rp = robotparser.RobotFileParser()
rp.set_url(URL.rsplit("/", 1)[0] + "/robots.txt")
rp.read()

rows = []
if rp.can_fetch(UA, URL): # (a) allowed?
    html = requests.get(URL, headers={"User-Agent": UA}, timeout=8).text
    soup = BeautifulSoup(html, "html.parser")
    for tr in soup.select("table tbody tr"): # (b) parse rows
        cells = tr.select("td")
        if len(cells) >= 2:
            rows.append({
                "currency": cells[0].get_text(strip=True),
                "rate": float(cells[1].get_text(strip=True)),
            })
```

`can_fetch(UA, URL)` gates the request; guard the cell count before indexing and cast the rate to float.

Mark scheme summary

Part	Coverage (notebooks)	Marks
A	Python foundations (NB 1–5)	20
B	Data, pandas, viz & statistics (NB 6–9)	20
C	ML, evaluation & feature engineering (NB 8, 16, 17)	20
D	AI engineering: LLM, RAG, agents, MCP, doc AI (NB 10–14, 18)	20
E	Applied & production: TS, NLP, deploy, scraping, capstone (NB 15–22)	20
Total		100

30 questions · MCQ, short-answer, write-the-code, debug, and one applied scenario per part · PayFlow fintech throughout.